
Analisi Tecnica del Firmware

Progetto *Vera-Module-RBL-XE*

Analisi del progetto a cura di RetroBit Lab

9 giugno 2026

Indice

1	Introduzione	1
2	Architettura del Sistema	1
2.1	Componenti Chiave	1
2.2	Modalità operative (BUS_MODE)	1
2.3	Ambienti di build PlatformIO	2
2.4	Interfacciamento Bus	2
3	Mapping dei Pin (ESP32-S3FN8, QFN56)	3
4	Analisi delle Tempistiche	4
4.1	Accesso diretto ai registri GPIO	5
4.2	Diagramma Temporale di un Ciclo di Lettura (PBI)	5
5	Frammento di Codice Sorgente	5
6	Diagramma di Flusso del Firmware	8
7	Conclusione	10

1 Introduzione

Questo documento presenta un'analisi tecnica del firmware per il progetto *Vera-Module-RBL-XE*. L'obiettivo del firmware, eseguito su un microcontrollore ESP32, è di interfacciarsi con il bus di un processore 6502 per emulare un dispositivo PBI (Parallel Bus Interface) o CCTL (Cartridge Control Line) per computer ATARI della serie XL/XE.

Il progetto supporta due MCU target e due modalità operative, per un totale di quattro varianti di compilazione distinte. L'analisi si concentra sulla fattibilità temporale (timing) dell'interazione tra la MCU moderna e la CPU vintage, illustrando la logica di funzionamento tramite diagrammi di flusso e temporali.

2 Architettura del Sistema

Il sistema si basa sull'interazione di due componenti principali.

2.1 Componenti Chiave

- **CPU 6502 (MOS/SALLY):** Il processore centrale dell'Atari, operante a una frequenza di clock di **1.7897 MHz** (NTSC).
- **MCU (selezionabile a compile-time):** Il microcontrollore moderno responsabile del monitoraggio del bus e dell'emulazione della periferica. Sono supportati due target:
 - **ESP32-S3FN8** (QFN56, 7×7 mm) — Dual-core Xtensa LX7 @ 240 MHz, 512 KB SRAM, 8 MB flash in-package Quad SPI, 45 GPIO bidirezionali.
 - **ESP32-WROOM-32 / NodeMCU DevKit V1** (ESP32-PICO-D4) — Dual-core Xtensa LX6 @ 240 MHz, 520 KB SRAM, 4 MB flash, GPIO34–39 solo input.

2.2 Modalità operative (BUS_MODE)

Il firmware supporta due modalità operative, selezionate a compile-time tramite la macro `BUS_MODE`:

- **PBI mode (BUS_MODE=0):** Il modulo si collega al connettore ECI della serie XL/XE.
 - **VCS latch:** scrittura di \$80 a \$D1FF attiva il dispositivo; qualsiasi altro valore lo disattiva.
 - **ROM \$D800–\$DFFF (2 KB, 11 bit A0–A10):** il segnale `ROM_SEL_N` (dalla 74HCT138 via 74LVC) attiva il serving. `MPD` (Math Pack Disable, GPIO42) viene asserito basso per ogni ciclo ROM con device selezionato, disabilitando la ROM Math Pack interna all'Atari.
 - **RAM \$D600–\$D7FF (512 B, 9 bit A0–A8):** il segnale `RAM_SEL_N` (GPIO40) attiva il buffer IRAM. `EXTSEL_N` viene asserito per-ciclo.

- **Registri VERA \$D100–\$D11F** (A0–A4, 32 registri) e **registri interni ESP32 \$D120–\$D1FE**: gestiti via D1XX_N. EXTSEL_N viene asserito per ogni ciclo in questo intervallo (escluso \$D1FF).
- **EXTSEL_N (GPIO0)**: segnale *per-ciclo* che disabilita MMU/Freddie dell'Atari per accessi D1xx e D6xx. Non è un segnale persistente.
- **CCTL mode (BUS_MODE=1)**: Il modulo si collega alla porta cartuccia. Il firmware decodifica i 5 bit bassi dell'indirizzo nell'area \$D500–\$D5FF tramite il segnale CCTL_N e genera il chip-select per il chip VERA.

2.3 Ambienti di build PlatformIO

Il file `platformio.ini` definisce quattro ambienti indipendenti:

Ambiente	MCU	Modalità	Piattaforma
<code>nodemcu-pbi</code>	ESP32-WROOM-32	PBI	espressif32@6.6.0
<code>nodemcu-cctl</code>	ESP32-WROOM-32	CCTL	espressif32@6.6.0
<code>esp32s3fn8-pbi</code>	ESP32-S3FN8	PBI	espressif32@6.6.0
<code>esp32s3fn8-cctl</code>	ESP32-S3FN8	CCTL	espressif32@6.6.0

La selezione dell'MCU avviene tramite le macro `MCU_ESP32_S3` e `MCU_ESP32_CLASSIC` passate come `-D` nei `build_flags`.

2.4 Interfacciamento Bus

L'ESP32 è collegato direttamente alle seguenti linee del bus del 6502 tramite tre level-shifter bidirezionali TXS0108E (3.3 V lato MCU, 5 V lato Atari):

- **Bus Dati (D0–D7)**: Bidirezionale; la direzione è controllata dal segnale `R/W_`.
- **Bus Indirizzi (A0–A10)**: Solo input; utilizzati per decodificare l'area di memoria di interesse. In modalità CCTL vengono usati solo A0–A4.
- **Segnali di controllo input**: `PHI2`, `R/W_`, `D1XX_N/CCTL_N`, `ROM_SEL_N`, `RAM_SEL_N` (GPIO40, solo PBI). I segnali `D1XX_N`, `ROM_SEL_N` e `RAM_SEL_N` provengono da logica combinatoria 74HCT (5 V) seguita da uno stadio 74LVC (3.3 V, ingressi 5V-tolerant): le uscite arrivano già a 3.3 V direttamente ai GPIO, senza level shifter intermedio.
- **Segnali di controllo output**: `EXTSEL_N` (GPIO0, per-ciclo, via U3), `DEV_SEL_N` (GPIO1, VERA chip select, diretto), `MPD` (GPIO42, Math Pack Disable, via U3 canale liberato).

3 Mapping dei Pin (ESP32-S3FN8, QFN56)

I pin fisici del package QFN56 sono numerati 1–56 (più il pad GND centrale, pin 57). La tabella seguente riporta i segnali del progetto con il numero GPIO logico e il numero di pad fisico del QFN56 (da Table 2-1 del datasheet ESP32-S3 v2.2). Il MCU opera con oscillatore RC interno a 240 MHz; nessun cristallo esterno è previsto.

Segnale	GPIO	Pin QFN56	IO MUX	Dir.	Note
<i>Bus dati D0–D7 (bank-0, bit 4–11) — bidirezionale via TXS0108E U1</i>					
D0	4	9	GPIO4	I/O	glitch LOW 60 µs al power-up
D1	5	10	GPIO5	I/O	glitch LOW 60 µs
D2	6	11	GPIO6	I/O	glitch LOW 60 µs
D3	7	12	GPIO7	I/O	glitch LOW 60 µs
D4	8	13	GPIO8	I/O	glitch LOW 60 µs
D5	9	14	GPIO9	I/O	glitch LOW 60 µs
D6	10	15	GPIO10	I/O	glitch LOW 60 µs
D7	11	16	GPIO11	I/O	glitch LOW 60 µs
<i>Bus indirizzi A0–A5 (bank-1, bit 1–6) — input via TXS0108E U2</i>					
A0	33	38	GPIO33	IN	
A1	34	39	GPIO34	IN	
A2	35	40	GPIO35	IN	
A3	36	41	GPIO36	IN	
A4	37	42	GPIO37	IN	
A5	38	43	GPIO38	IN	
<i>Bus indirizzi A6–A10 (bank-0) — input via TXS0108E U2/U3, solo PBI</i>					
A6	12	17	GPIO12	IN	via U2 — glitch LOW 60 µs
A7	13	18	GPIO13	IN	via U2 — glitch LOW 60 µs
A8	14	19	GPIO14	IN	via U3 — glitch LOW 60 µs
A9	15	21	XTAL_32K_P	IN	via U3 — pad RTC usato come G
A10	16	22	XTAL_32K_N	IN	via U3 — pad RTC usato come G
<i>Segnali di controllo input — PHI2 e R/W_ via U3; D1XX_N, ROM_SEL_N, RAM_SEL_N diretti</i>					
PHI2	2	7	GPIO2	IN	via U3 — glitch LOW 60 µs
R/W_	17	23	GPIO17	IN	via U3 — glitch LOW 60 µs
D1XX_N	18	24	GPIO18	IN	74HCT+74LVC, uscita 3.3 V diret
ROM_SEL_N	21	27	GPIO21	IN	74HCT138+74LVC, uscita 3.3 V d
RAM_SEL_N	40	45	MTDO	IN	74HCT+74LVC, uscita 3.3 V diret
<i>Segnali di controllo output</i>					
EXTSEL_N	0	5	GPIO0	OUT	via U3 A8/B8 — per-ciclo (D1xx
DEV_SEL_N	1	6	GPIO1	OUT	diretto a VERA CS (3.3 V); glitch
MPD	42	48	MTMS	OUT	via U3 A6/B6 — Math Pack Disa
<i>Debug</i>					
UART TX	43	49	U0TXD	OUT	Serial debug @ 115200 baud
UART RX	44	50	U0RXD	IN	non utilizzato in firmware

Indirizzamento memoria

`decode_addr()` restituisce A0–A10 (11 bit). Le maschere di accesso ai buffer IRAM sono:

ROM \$D800–\$DFFF : `pbi_driver[addr & 0x7FF]` (2048 B, 11 bit A0–A10)

RAM \$D600–\$D7FF : `ram_pbi[addr & 0x1FF]` (512 B, 9 bit A0–A8)

GPIO disponibili

I seguenti GPIO non sono assegnati al progetto e sono liberi per usi futuri:

GPIO	Pin QFN56	IO MUX	Note
19	25	USB_D–	Libero su PCB custom (USB CDC disabilitato)
20	26	USB_D+	Libero su PCB custom (USB CDC disabilitato)
39	44	MTCK	JTAG clock — usabile come GPIO
41	46	MTDI	JTAG data in — usabile come GPIO
44	50	U0RXD	UART RX non usata
47	53	FSPICLK	GPIO generico
48	54	FSPID	GPIO generico

Attenzione: I pin QFN56 28 e 30–35 (GPIO26–32) sono collegati internamente alla flash Quad SPI in-package (variante FN8). Non devono mai essere connessi a segnali esterni. GPIO3 (pin 8) è pin di strapping JTAG senza pull interno: va collegato a GND tramite 10 kΩ. GPIO45 e GPIO46 sono pin di strapping: lasciare liberi.

4 Analisi delle Tempistiche

Una delle principali preoccupazioni in un progetto di questo tipo è la capacità della MCU di rispondere entro la finestra temporale definita dal ciclo di clock della CPU 6502.

- **Ciclo Clock 6502 (1.7897 MHz, NTSC):** $T_{6502} = 1/(1.7897 \times 10^6) \approx 559 \text{ ns}$
- **Finestra PHI2 alto (half-cycle):** $T_{PHI2\uparrow} \approx 279 \text{ ns}$ — questa è la finestra critica entro cui l'ESP32 deve leggere l'indirizzo, decodificarlo ed eventualmente pilotare il bus dati.
- **Ciclo Clock ESP32 (240 MHz):** $T_{ESP32} = 1/(240 \times 10^6) \approx 4.17 \text{ ns}$

Il rapporto tra la finestra critica e il ciclo dell'ESP32 è:

$$\frac{T_{PHI2\uparrow}}{T_{ESP32}} = \frac{279 \text{ ns}}{4.17 \text{ ns}} \approx 67 \text{ cicli ESP32}$$

Anche considerando solo la semi-finestra di PHI2 alto (il caso peggiore), l'ESP32-S3 dispone di circa **67 cicli di clock** per completare tutta la catena: lettura di `GPIO.in` / `GPIO.in1.val`, decodifica dell'indirizzo, lookup nella LUT, pilotaggio del bus dati. Poiché tutte le funzioni critiche risiedono in IRAM (`IRAM_ATTR`) ed operano solo su registri hardware, il percorso critico è tipicamente inferiore a 20 cicli.

4.1 Accesso diretto ai registri GPIO

Per eliminare qualsiasi overhead di astrazione, il firmware usa esclusivamente accesso diretto ai registri hardware dell'ESP32:

- `GPIO.in` — legge i 32 GPIO del bank-0 in un'unica istruzione a 32 bit.
- `GPIO.in1.val` — legge i GPIO del bank-1 (GPIO32–48).
- `GPIO.out_w1ts` / `GPIO.out_w1tc` — set/clear atomico dei bit di uscita (write-1-to-set / write-1-to-clear).
- `GPIO.enable_w1ts` / `GPIO.enable_w1tc` — abilitazione/disabilitazione in uscita (tristate) per il bus dati.

4.2 Diagramma Temporale di un Ciclo di Lettura (PBI)

Il diagramma seguente illustra un tipico ciclo di lettura gestito dall'ESP32 in modalità PBI.

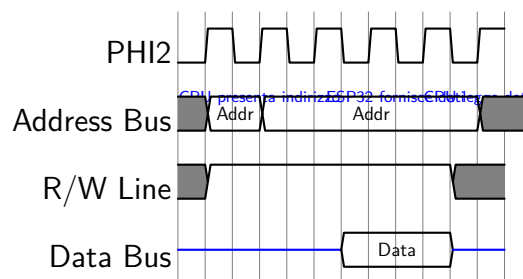


Figura 1: Diagramma temporale di un ciclo di lettura del 6502 gestito dall'ESP32 (modalità PBI).

5 Frammento di Codice Sorgente

Di seguito è riportato un estratto del file `main.cpp` che mostra la struttura di selezione a compile-time della MCU e il cuore del `MonitorTask` in modalità PBI. Tutte le funzioni critiche sono marcate `IRAM_ATTR` per garantire l'esecuzione dalla RAM interna, indipendentemente dallo stato della cache flash.

```
1 #if defined(MCU_ESP32_S3)
2 #define PIN_PHI2      2      /* bank-0 */
3 #define PIN_RW        17
4 #define PIN_BUS_SEL_N 18      /* D1XX_N, da 74HCT+74LVC diretta 3.3V */
5 #define PIN_ROM_SEL_N 21      /* da 74HCT138+74LVC, diretta 3.3V */
6 #define PIN_RAM_SEL_N 40      /* MTDO, QFN56 pin45, bank-1 bit8 */
7 #define PIN_EXTSEL_N   0      /* output, strapping, pull-up 10kohm */
8 #define PIN_DEV_SEL_N  1      /* VERA chip select, diretto */
9 #define PIN_MPD        42      /* MTMS, QFN56 pin48, bank-1 bit10 */
10 #define PIN_UART_TX    43
11
```

```

12  /* MPD su bank-1: usare out1_wlts/out1_wlts */
13  #define MPD_ASSERT()    GPIO.out1_wlts.val = (1UL << (PIN_MPD - 32))
14  #define MPD_DEASSERT()  GPIO.out1_wlts.val = (1UL << (PIN_MPD - 32))
15  /* RAM_SEL_N su bank-1 */
16  #define DECODE_IS_RAM(hi) (((hi) & (1UL << (PIN_RAM_SEL_N-32))) == 0)
17
18  #define DBUS_MASK    0x00000FF0UL    /* GPIO4-11, bank-0 bit 4-11 */
19
20  /* A0-A5: GPIO.in1.val bit 1-6; A6-A10: GPIO.in bit 12-16 */
21  static inline uint16_t IRAM_ATTR decode_addr(uint32_t lo, uint32_t hi)
22  {
23      uint16_t a = (uint16_t)((hi >> 1) & 0x3Fu); /* A0-A5 GPIO33-38 */
24      a |= (uint16_t)((lo >> 12) & 1u) << 6;      /* A6      GPIO12      */
25      a |= (uint16_t)((lo >> 13) & 1u) << 7;      /* A7      GPIO13      */
26      a |= (uint16_t)((lo >> 14) & 1u) << 8;      /* A8      GPIO14      */
27      a |= (uint16_t)((lo >> 15) & 1u) << 9;      /* A9      GPIO15      */
28      a |= (uint16_t)((lo >> 16) & 1u) << 10;     /* A10     GPIO16      */
29      return a;
30  }
31  static inline uint8_t IRAM_ATTR decode_data(uint32_t lo) {
32      return (uint8_t)((lo >> 4) & 0xFFu); /* GPIO4-11 compatti */
33  }
34  #elif defined(MCU_ESP32_CLASSIC)
35  /* NodeMCU / PICO-D4: nessun PIN_RAM_SEL_N ne' PIN_MPD disponibili */
36  #warning "MCU_ESP32_CLASSIC: RAM $D600-$D7FF e MPD non supportati"
37  #define MPD_ASSERT()    /* no-op */
38  #define MPD_DEASSERT()  /* no-op */
39  #define DECODE_IS_RAM(hi)    false
40  /* D0-D7: GPIO18,19,21-23,25-27 (non contigui in bank-0) */
41  static inline uint8_t IRAM_ATTR decode_data(uint32_t lo) {
42      return (uint8_t)((lo >> 18) & 0x03u) |
43              (((lo >> 21) & 0x07u) << 2) |
44              (((lo >> 25) & 0x07u) << 5));
45  }
46  #else
47  #error "Set -D MCU_ESP32_S3 or -D MCU_ESP32_CLASSIC"
48  #endif

```

Listing 1: Definizioni pin e macro MCU-specifiche per ESP32-S3FN8

```

1  static void IRAM_ATTR MonitorTask(void *arg)
2  {
3      bool selected = false;
4      GPIO.out_wlts = (1UL << PIN_EXTSEL_N) | (1UL << PIN_DEV_SEL_N);
5      MPD_DEASSERT();
6      bus_release(); /* tristate data bus */
7
8      for (;;) {
9          /* 1. Attesa fronte di salita PHI2; snapshot bank-0 e bank-1 */
10         uint32_t g_lo;
11         while (!((g_lo = GPIO.in) & (1UL << PIN_PHI2)));
12         uint32_t g_hi = GPIO.in1.val;
13
14         /* 2. Decodifica segnali e indirizzo (A0-A10) */
15         bool is_read = (g_lo & (1UL << PIN_RW)) != 0;

```



```

16     bool is_d1xx = (g_lo & (1UL << PIN_BUS_SEL_N)) == 0;
17     bool is_rom  = (g_lo & (1UL << PIN_ROM_SEL_N))  == 0;
18     bool is_ram  = DECODE_IS_RAM(g_hi);
19     uint16_t addr = decode_addr(g_lo, g_hi);
20     uint8_t  off  = (uint8_t)(addr & 0xFFu);
21     bool is_d1ff      = is_d1xx && (off == 0xFFu);
22     bool is_vera_range = is_d1xx && (off <= 0x1Fu);
23     bool is_int_range  = is_d1xx && (off >= 0x20u) && (off <= 0xFEu)
;

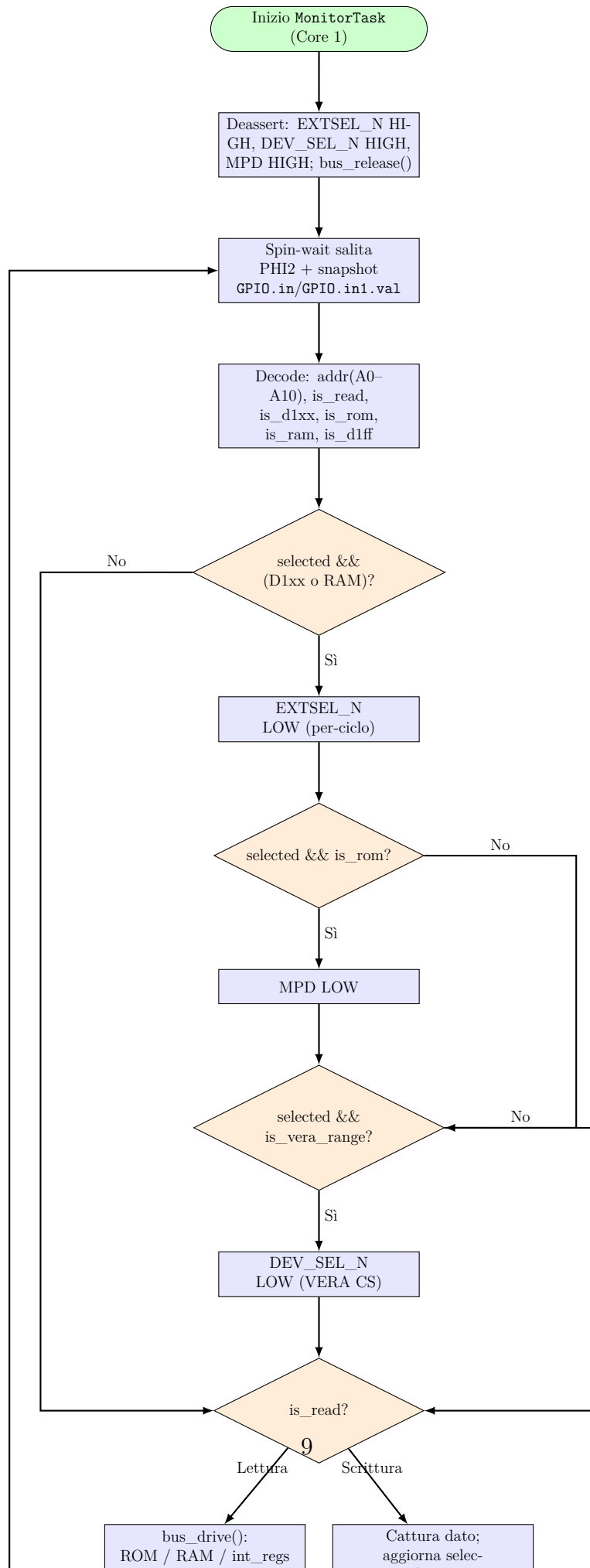
24
25     /* 3. EXTSEL_N per-ciclo: D1xx (escluso D1FF) e RAM D6xx */
26     if (selected && ((is_d1xx && !is_d1ff) || is_ram))
27         GPIO.out_w1tc = (1UL << PIN_EXTSEL_N);
28
29     /* 4. MPD solo per ROM $D800-$DFFF con device selezionato */
30     if (selected && is_rom)
31         MPD_ASSERT();
32
33     /* 5. VERA CS per range $D100-$D11F (R e W) */
34     if (selected && is_vera_range)
35         GPIO.out_w1tc = (1UL << PIN_DEV_SEL_N);
36
37     /* 6. Ciclo di LETTURA */
38     if (is_read && selected) {
39         if (is_rom)          bus_drive(pbi_driver[addr & 0x7FFu])
;
40         else if (is_ram)     bus_drive(ram_pbi    [addr & 0x1FFu])
;
41         else if (is_int_range) bus_drive(int_regs [off]);
42         /* is_vera_range: VERA guida il bus autonomamente */
43     }
44     /* 7. Ciclo di SCRITTURA */
45     else if (!is_read) {
46         uint8_t data = decode_data(GPIO.in);
47         if (is_d1ff) {
48             bool new_sel = (data == 0x80u); /* VCS latch */
49             if (new_sel != selected) {
50                 selected = new_sel;
51                 xQueueSend(eventQueue, &selected, 0);
52             }
53         } else if (selected && is_int_range) {
54             int_regs[off] = data;
55         }
56     }
57
58     /* 8. Fine ciclo: rilascia bus, deasserta tutti gli output */
59     while (GPIO.in & (1UL << PIN_PHI2));
60     bus_release();
61     MPD_DEASSERT();
62     GPIO.out_w1ts = (1UL<<PIN_EXTSEL_N)|(1UL<<PIN_DEV_SEL_N);
63 }
64 }

```

Listing 2: Nucleo del MonitorTask (modalità PBI, semplificato)

6 Diagramma di Flusso del Firmware

La logica principale del firmware è contenuta nel `MonitorTask`, che viene eseguito sul secondo core dell'ESP32 (Core 1) con priorità massima per garantire prestazioni in tempo reale.



7 Conclusione

L'analisi conferma che l'architettura hardware e software del progetto è solida e performante. L'ESP32-S3FN8 a 240 MHz garantisce circa 67 cicli di clock nella finestra critica di PHI2 alto (279 ns), ampiamente sufficienti per l'intera catena di decodifica e pilotaggio del bus. Le tecniche di programmazione a basso livello — accesso diretto ai registri GPIO (GPIO.in, GPIO.in1.val, GPIO.out_w1ts/w1tc, GPIO.out1_w1tc/out1_w1ts), funzioni in IRAM, LUT pre-calcolata per il bus dati — garantiscono un percorso critico tipicamente inferiore a 20 cicli.

Segnali di controllo revisionati

La revisione del firmware e della mappatura hardware ha chiarito e corretto i seguenti aspetti:

- **EXTSEL_N** è un segnale *per-ciclo*: viene asserito basso solo durante i cicli in cui il device è selezionato (`selected=true`) e l'accesso è nell'intervallo \$D100–\$D1FE (escluso \$D1FF) oppure nella RAM interna \$D600–\$D7FF. Viene sempre deassertato al termine del ciclo. Non è un segnale persistente legato allo stato del latch VCS.
- **MPD (Math Pack Disable)** viene asserito basso esclusivamente quando il device è selezionato e l'accesso riguarda la ROM interna \$D800–\$DFFF. Non viene asserito per accessi alla RAM, ai registri interni o a VERA. È connesso al GPIO 42 (MTMS, QFN56 pin 48) nel bank-1 e utilizza `GPIO.out1_w1tc.val` per l'asserzione.
- **RAM interna 512 B (\$D600–\$D7FF)**: indirizzata su 9 bit via `ram_pbi[addr & 0x1FFu]`. Il segnale `RAM_SEL_N` è connesso al GPIO 40 (MTDO, QFN56 pin 45, bank-1 bit 8).
- **Percorso 74HCT + 74LVC**: i segnali `D1XX_N`, `ROM_SEL_N` e `RAM_SEL_N` sono generati da IC 74HCT (5 V) e portati a livello 3.3 V da uno stadio 74LVC con ingressi 5V-tolerant. Le uscite 3.3 V si collegano *direttamente* ai GPIO dell'ESP32-S3 senza ulteriori level shifter (TXS0108E).
- **Canali U3 liberati**: poiché `D1XX_N` e `ROM_SEL_N` non passano più per U3 (TXS0108E), i canali A6/B6 e A7/B7 risultano disponibili. Il canale A6/B6 è ora assegnato a MPD (GPIO 42, output ESP32 → bus Atari 5 V); A7/B7 rimane di riserva.
- **GPIO 15 e GPIO 16** (pad RTC XTAL_32K_P/N): il progetto non impiega quarzo da 32 kHz; il MCU opera con oscillatore interno RC a 240 MHz. I pad sono utilizzati come normali GPIO per A9 e A10.

Il supporto dual-MCU (MCU_ESP32_S3 / MCU_ESP32_CLASSIC) e dual-mode (BUS_MODE=0 PBI, BUS_MODE=1 CCTL) rende il firmware portabile su hardware diverso senza duplicazione di codice. La tabella dei pin fisici del package QFN56 consente di realizzare il PCB direttamente dalla documentazione senza ambiguità tra numero GPIO logico e pad fisico.